

BIG DATA TECHNOLOGIES - CIE III

Student Name: Rahul Sharma USN: 1RV22AI045 Date: 17-06-2026

Max Marks: 50 Duration: 120 minutes

PART - A

Q.1.1 [2 Marks]

Two delivery guarantees provided by Apache Flume are:

1. End-to-End Delivery - Flume ensures data is delivered from source to destination reliably.
2. Transactional Reliability (Guaranteed Delivery) - Flume uses a transaction-based approach between Source-Channel and Channel-Sink to ensure reliable data transfer.

Q.1.2 [2 Marks]

Wide transformations are transformations where data from multiple partitions needs to be shuffled across the cluster. Examples include `groupByKey()`, `reduceByKey()`, and `join()`.

These transformations require data movement across nodes.

Q.1.3 [2 Marks]

The two processing models of Apache Spark are:

1. Batch Processing - processes large volumes of stored data at once
2. Stream Processing (Spark Streaming) - processes real-time data streams continuously

Q.1.4 [2 Marks]

An interceptor is a type of data storage mechanism in Hadoop that stores intermediate results during MapReduce operations. It acts as a buffer between the mapper and reducer phases.

Q.1.5 [2 Marks]

The two categories of operations on RDDs in Spark are:

1. Transformations - These create new RDDs from existing ones (e.g., `map`, `filter`, `flatMap`). They are lazily evaluated.
2. Actions - These trigger computation and return results to the driver program (e.g., `count`, `collect`, `reduce`).

PART - B

Q.1a [5 Marks]

RDD (Resilient Distributed Dataset) is the fundamental data abstraction in Apache Spark. It is an immutable, distributed collection of objects.

Method 1: Parallelizing an Existing Collection

Spark can create an RDD from a local collection using the `parallelize()` method.

```
val data = Array(1, 2, 3, 4, 5)
```

```
val rdd = sc.parallelize(data)
```

The collection is divided into partitions and distributed across the cluster nodes. This is useful for testing with small datasets.

Method 2: Loading from External Sources

RDDs can also be created from files stored externally.

Q.1b [5 Marks]

Data serialization in Apache Spark is the process of converting objects into a byte stream for efficient transmission and storage across the cluster.

1. Java Serialization:

- This is the default serialization mechanism in Spark.
- It is easy to use as it works with any class that implements `java.io.Serializable`.
- However, it is slower and produces larger serialized objects.

2. Kryo Serialization:

- Kryo is a faster and more compact serialization library.
- It is recommended for large-scale Spark applications where performance matters.
- Configuration: `sparkConf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")`
- Kryo reduces network traffic significantly and improves overall performance.
- Custom classes need to be registered with Kryo for optimal performance.

Q.2a [6 Marks]

Apache Flume is a distributed system for collecting and moving large amounts of log data.

Working of Flume:

1. The Source component receives data from external sources.
2. Events are written to a Channel for temporary storage.
3. The Sink reads events from the Channel and writes to the destination.

[Diagram not drawn]

Code Snippet:

```
agent.sources=s1
agent.channels=c1
agent.sinks=k1
agent.sources.s1.type=spooldir
agent.sources.s1.spoolDir=/tmp/logs
agent.channels.c1.type=memory
agent.sinks.k1.type=logger
agent.sources.s1.channels=c1
agent.sinks.k1.channel=c1
```

Q.2b [4 Marks]

Apache Flume is a tool used in the Hadoop ecosystem. It can be used to move data from one place to another. Flume has sources, channels, and sinks. It is useful for big data processing and can handle log files.

Q.3a [6 Marks]

A two-tier Flume configuration uses two Flume agents arranged in sequence.

Agent 1 (Collection Tier):

- Source: Spooling Directory Source that monitors a directory for new files
- Channel: Memory Channel
- Sink: Avro Sink that forwards events to Agent 2

Agent 2 (Storage Tier):

- Source: Avro Source that receives events from Agent 1

- Channel: Memory Channel
- Sink: HDFS Sink

The spooling directory source reads files from a directory and converts them to Flume events. After processing, files are marked as completed.

Configuration for Agent 1:

```
agent1.sources = spoolSrc
agent1.channels = memChannel
agent1.sinks = avroSink
agent1.sources.spoolSrc.type = spooldir
agent1.sources.spoolSrc.spoolDir = /data/input
```

Q.3b [4 Marks]

Integrating Flume with Applications:

Apache Flume can be integrated directly with applications to collect and transport event data into Hadoop, rather than just monitoring files or network streams.

Methods of Integration:

1. RPC Client:

- The application uses Flume's RPC (Remote Procedure Call) client library.
- Events are created within the application and sent to a Flume agent through an Avro RPC connection.
- The Flume agent processes and stores the events in destinations like HDFS or HBase.

2. Embedded Agent:

- Flume provides an embedded agent that runs inside the application process itself.
- The application sends events directly to the embedded agent.
- The agent uses configured sources, channels, and sinks to deliver events to the final destination.
- This eliminates the need for a separate Flume process.

Q.4a [6 Marks]

Shared Variables in Spark:

By default, Spark sends a copy of variables to each task. Shared variables allow efficient data sharing across tasks.

There are two types:

1. Broadcast Variables:

A broadcast variable is a read-only variable cached on each worker node rather than shipped with every task.

- Created using `SparkContext.broadcast()`
- Distributed to all executors only once
- Tasks access the local copy using `.value`
- Reduces network communication

Example:

```
val lookupTable = sc.broadcast(Array("India", "USA", "UK"))
val result = rdd.filter(x => lookupTable.value.contains(x))
```

2. Accumulators:

Accumulators are variables used for aggregating values from worker nodes. They are like counters that can be updated by tasks.

Q.4b [4 Marks]

`reduceByKey()`:

This transformation aggregates values that have the same key using an associative and commutative reduce function.

Example:

```
val data = sc.parallelize(List(("A",1), ("A",2), ("B",3)))
```

```
data.reduceByKey(_ + _)
```

Output: (A,3), (B,3)

`foldByKey()`:

Similar to `reduceByKey()` but takes an initial zero value. The zero value is applied to each partition independently.

Example:

```
data.foldByKey(10)(_ + _)
```

Output: (A,13), (B,13)

The key difference is that `foldByKey` starts aggregation with an initial value, while `reduceByKey` starts directly with the first element.

Q.5 [10 Marks]

Job Submission Phase in Spark:

A Spark job is submitted when an action like `count()`, `collect()`, or `saveAsTextFile()` is called on an RDD. The `SparkContext` submits the job to the scheduler in the driver program.

Job Submission Process:

1. When an action is invoked, Spark calls `runJob()` on the `SparkContext`.
2. The request is passed to the Scheduler in the Driver.
3. The scheduler has two components: DAG Scheduler and Task Scheduler.
4. The DAG Scheduler divides the job into stages.
5. The Task Scheduler assigns tasks to available executors.
6. The Cluster Manager allocates resources.
7. Tasks execute on worker nodes and return results.

[Diagram not drawn due to time constraint]

DAG Construction:

Spark converts RDD transformations into a Directed Acyclic Graph (DAG). The DAG helps Spark optimize the execution plan.